

Endpointuri care lipsesc — academy-hub-api

Ce ai scris și nu e accesibil prin HTTP, în ordinea în care merită expus.

Unde ești acum: ai scris **25 de metode** în cele patru query service-uri și **4** sunt accesibile prin HTTP (`GET /api/students` , `/courses` , `/books` , `/enrollments`). Celelalte 21 sunt cod pe care nu-l cheamă nimeni: nu le execută nimic, deci nimic nu le observă defectele. La runda 3 aceeași situație era la command service-uri, iar bug-ul din `createBook` a supraviețuit o rundă întreagă exact de asta.

Fișa asta le ia pe rând. **Ordinea contează** — partea 1 e cea care lipsește cel mai tare.

Repară întâi 🚫 B1 din code review (aplicația nu pornește pe o bază nouă), altfel nu poți testa nimic din ce scrii aici pe o bază curată.

Partea 1 — Resursa individuală

Momentan **nu poți citi un singur student**. `GET /api/students/{id}` răspunde 405: calea există (o mapează `PUT` și `DELETE`), verbul nu. Poți șterge un student, dar nu-l poți vedea. Niciuna din cele patru resurse n-are `GET /{id}` .

#	Endpoint	Ce întoarce	Serviciul există deja
1	<code>GET /api/students/{id}</code>	un student, cu cărțile și cursurile lui	<code>StudentQueryService.getStudentById</code>
2	<code>GET /api/courses/{id}</code>	un curs, cu numărul de înscriși	<code>CourseQueryService.findById</code>
3	<code>GET /api/books/{id}</code>	o carte, cu numele studentului	— de adăugat (<code>BookRepository.findById</code>)
4	<code>GET /api/enrollments/{id}</code>	o înscriere	— de adăugat (<code>EnrollmentRepository.getById</code>)

Gata când: pentru fiecare, un id real întoarce **200** cu obiectul, iar `00000000-0000-0000-0000-000000000000` întoarce **404**, nu 500. (404-ul cere advice-ul din  B3 — cele două teme se închid împreună.)

Partea 2 — Cărțile și înscrierile unui student

Ai scris `getStudentBooks(UUID)` și `countBooksByStudentId(UUID)`, plus interogarea cu `join fetch` pe cărți. Nimic din ele nu e accesibil. Un client care vrea cărțile unui student trebuie să ceară toată lista de studenți și să caute singur.

#	Endpoint	Ce întoarce	Serviciul există deja
5	<code>GET /api/students/{id}/books</code>	cărțile studentului	<code>BookQueryService.getStudentBooks</code>
6	<code>GET /api/students/{id}/books/count</code>	câte cărți are	<code>BookQueryService.countBooksByStudentId</code>
7	<code>GET /api/students/{id}/enrollments</code>	înscrierile studentului	— de adăugat
8	<code>GET /api/courses/{id}/enrollments</code>	studenții înscriși la curs	— de adăugat

Gata când: `GET /api/students/{id}/books` pe un student fără nicio carte întoarce **200** `[]` — nu 500, nu 404. Studentul există, lista lui e goală; sunt două lucruri diferite. (Se leagă de  B8.)

Partea 3 — Filtre

Cinci metode de căutare scrise și niciun mod de a le chema. Filtrele nu sunt căi noi, sunt parametri pe lista pe care o ai deja: `GET /api/students?minAge=20`.

#	Endpoint	Ce întoarce	Serviciul există deja
9	<code>GET /api/students?minAge=</code>	studenții peste vârsta dată	<code>getStudentsWithAgeGreaterThan</code>
10	<code>GET /api/students?maxAge=</code>	studenții sub vârsta dată	<code>getStudentsWithAgeLessThan</code>
11	<code>GET /api/students?firstName=</code>	studenții cu prenumele dat, sortați pe vârstă	<code>findByFirstNameOrderByAgeAsc</code>
12	<code>GET /api/students?email=</code>	studentul cu emailul dat	<code>getByEmail</code>
13	<code>GET /api/courses?department=</code>	cursurile departamentului	<code>findByDepartment</code>
14	<code>GET /api/courses?department=&sort=name</code>	aceleași, sortate pe nume	<code>findByDepartmentOrderByNameAsc</code>

Gata când: `GET /api/students` fără niciun parametru întoarce în continuare toată lista, iar fiecare parametru o restrânge. Un parametru necunoscut nu strică cererea.

⚠ Verifică nr. 14 **prin rulare**, uitându-te la ordinea din răspuns: metoda ta `findByDepartmentOrderByNameAsc` nu mai sortează (M2 din code review). Numele promite o ordine pe care corpul metodei n-o mai cere.

Partea 4 — Statistici

Două proiecții scrise cu `@Query`, ambele inaccesibile. Sunt cele mai interesante date din aplicația ta și singurele pe care un client nu și le poate calcula singur fără să descarce totul.

#	Endpoint	Ce întoarce	Serviciul există deja
15	<code>GET /api/courses/stats/per-department</code>	câte cursuri are fiecare departament	<code>findAndCountPerDepartment</code>
16	<code>GET /api/students/stats/book-counts</code>	câte cărți are fiecare student	<code>getStudentBookCounts</code>
17	<code>GET /api/students/stats/top-by-books</code>	studenții ordonați descrescător după cărți	<code>getStudentsOrderByBooksDesc</code>
18	<code>GET /api/students/oldest</code>	cel mai vârstnic student	<code>getOldestStudent</code>
19	<code>GET /api/courses/{id}/enrollments/count</code>	câți studenți are cursul	<code>countByDepartment</code> e pe departament — asta e de adăugat

Gata când: pe baza de seed, nr. 15 întoarce un rând pentru fiecare departament distinct, iar suma numerelor e egală cu ce-ți dă `GET /api/courses`.

Partea 5 — Ce lipsește la scriere

CRUD-ul e complet ca verbe, dar două lucruri nu stau cum trebuie.

#	Cerință	De ce
20	<code>DELETE /api/enrollments/{id}</code>	acum ștergerea ia id-urile din corpul cererii; multe proxy-uri și clienți HTTP aruncă corpul unui DELETE (M6 din code review)
21	<code>DELETE /api/students/{sid}/enrollments/{cid}</code>	varianta „scoate studentul de la cursul ăsta”, dacă vrei să păstrezi ștergerea după pereche — dar cu id-urile în cale, nu în corp
22	<code>POST /api/students/{id}/books</code>	crearea unei cărți e o operație pe un student (cartea nu există fără proprietar); <code>POST /api/books</code> cu <code>studentId</code> în corp merge, dar calea nu spune cine e proprietarul

Gata când: vechiul `DELETE /api/enrollments` cu corp nu mai e singura cale de ștergere, iar noua variantă întoarce **404** pentru o înscriere inexistentă.

Cum verifici tot, la final

Nu bifa nimic din citit. Pentru fiecare endpoint scris, trei cereri:

1. **cazul bun** — date reale → 200, și te uiți la corpul răspunsului, nu doar la cod
2. **cazul gol / inexistent** — id care nu există → 404; listă goală → 200 `[]`
3. **cazul stricat** — id care nu e UUID, parametru fără valoare → 400

Dacă vreuna din cele trei întoarce **500**, endpoint-ul nu e gata — oricât de corect răspunde pe cazul bun.

Reper: după fișa asta ar trebui să ai peste 20 de `@GetMapping` și niciun serviciu nechemat de nimeni. Când ajungi acolo, `Runner.java` nu mai are nicio justificare.

Ordinea pe care ți-o recomand

1. 🟡 **B1** din code review — altfel nu poți porni pe o bază curată și nimic din ce urmează nu se poate testa cinstit
2. 🟡 **B3** (advice-ul) — fără el fiecare endpoint nou pe care îl scrii aici va răspunde 500 în loc de 404, și vei crede că l-ai scris greșit
3. **Partea 1** — resursa individuală; e ce lipsește cel mai tare dintr-un API REST
4. **Partea 2**, apoi **3**, apoi **4** — în ordinea asta
5. **Partea 5** la final, e cea mai puțin urgentă

Nu le fă pe toate într-un commit. Un endpoint, o probă cu `curl`, apoi următorul.



MyCodeSchool Material didactic MyCodeSchool mycodeschool.ro